# ATOMIC-LEVEL MATERIALS SIMULATION

**Module Code: MES306TC**

Kenan Song
School of Chips
Email:
Kenan.Song@xjtlu.edu.cn
Office: C5007

# Density of State II

**I. Reciprocal Space Discretization: Monkhorst *k* Point Sampling**

**II. The 4 Foundational Mathematical Steps of TB-DOS calculation**

**III. Statistical Bin-Counting, Normalization and Grid Convergence Testing**

**IV. Projected DOS (PDOS) and Wavefunction Decomposition**

# I. Reciprocal Space Discretization: Monkhorst *k* Point Sampling

**Mapping out the Phase Space for Electronic Counting**

**Moving from Real Space vectors ($a_1$, $a_2$, $a_3$) to Reciprocal Space vectors ($b_1$, $b_2$, $b_3$)**

**Constructing the Hamiltonian $H(k)$ via phase factor modulations: $H_{ij}(K) = \sum_R t_{ij}(R) e^{ik \cdot R}$**

**The challenge: Evaluating this expression continuously across an entire 3*D* or 2*D* geometric volume**

**Before counting states, one must establish the grid. In real space, the lattice constants define the unit cell. In reciprocal space, these yield the reciprocal lattice vectors $b_i$. For every point *k*, choose in this reciprocal space, modulate the real space hopping parameters $t_{ij}(R)$ by a phase factor $e^{ik \cdot R}$ to construct a complex Hermitian matrix $H(k)$. Because calculating an infinite number of *k* points is impossible, one must discretize this continuous domain into a finite, highly representative numerical grid**

# I. Reciprocal Space Discretization: Monkhorst *k* Point Sampling

**Sampling Paradigms: Band Structures vs. DOS**

**Band Structure Calculation: Evaluates $H(k)$ strictly along 1D high-symmetry lines (*e.g.* $\Gamma \rightarrow \mathrm{K} \rightarrow \mathrm{M} \rightarrow \Gamma$)**

**Flaw for DOS: Completely misses the vast off-axis interior volume of the Brillouin Zone**

**DOS Calculation: Requires Full Brillouin Zone (Full-BZ) uniform sampling**

**The Rule: DOS is a statistical volume property; every single patch of phase space must have an equal probability of being counted**

**When plotting a band structure, one draw a path along high-symmetry lines, like $\Gamma \rightarrow \mathrm{K} \rightarrow \mathrm{M}$. This is perfect for seeing the band gap or Dirac cones, but it is completely useless for DOS.**

**Why? Because a 1D line has zero volume in a 2D or 3D space. It completely ignores the states residing in the interior regions of the zone. Because DOS is a global, integrated statistical property, one cannot just walk along a scenic path; it is necessary to sample the entire terrain uniformly using a dense grid**

# I. Reciprocal Space Discretization: Monkhorst *k* Point Sampling

**The Monkhorst-Pack Sampling Scheme:**

**A systematic method for generating an un-biased, uniformly spaced grid of *k* points inside the Brillouin Zone**

**Mathematical Form (Fractional Coordinates):**

$$\boldsymbol{k_{p,q,r} = u_p b_1 + v_q b_2 + w_r b_3}$$
$$\boldsymbol{u_p = \frac{2p - N_x - 1}{2N_x}, p = 1, 2, \cdots, N_x}$$

**Ensures the grid is centered, balanced, and maps perfectly to the crystal's periodic boundary conditions**

**To achieve this uniform distribution, one relies heavily on the Monkhorst-Pack scheme.**

**This algorithm generates a homogeneous grid of *k* points along the reciprocal lattice vectors. By using fractional coordinates shifted by half a grid spacing, it ensures that the points are distributed symmetrically around the origin. This balances the grid across the Brillouin Zone and aligns perfectly with the periodic boundaries of the crystal lattice**

# I. Reciprocal Space Discretization: Monkhorst *k* Point Sampling

**The grid density is designated by a mesh size: $N_x \times N_y \times N_z$**

**Total sampled points: $N_k = N_x \cdot N_y \cdot N_z$**

**Convergence Criteria: Increase the mesh size until the total integrated DOS up to the Fermi level, or the shape of the Van Hove peaks, stops changing**

**Rule of Thumb: DOS requires an order of magnitude more *k* points than self-consistent ground state energy calculations**

**Whenever running a calculation, one must perform a convergence test. One cannot simply guess a mesh size like 10×10×1 and assume the results are correct. Systematically ramp up the density—from 20×20 to 50×50, up to 100×100 or more—until the DOS profile stabilizes. Because DOS relies on capturing sharp features and sudden slope changes like Van Hove singularities, it demands a much higher density of *k* points than a standard ground-state total energy calculation**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Step 1: Grid Generation → Generates *k* mesh inside the First BZ**

↓

**Step 2: Diagonalization → Solves $H(k)|\Psi\rangle = E|\Psi\rangle$ at every single *k* point**

↓

**Step 3: Binning/Statistics → Sorts and counts energies into a uniform histogram**

↓

**Step 4: Normalization → Scales intensity against total states and volume**

**Let us map out the core engineering sequence of our code.**

**The execution pipeline follows four precise sequential stages:**

**1. Generate our grid coordinates**

**2. pass those coordinates into our matrix diagonalizer**

**3. group the resulting eigenvalues into distinct bins**

**4. scale the final counts so they match real physical units. Let's look at each step in detail**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Step 1: *k* Point Generation and Array Allocation**

**Loop over indices (*i*, *j*, *k*) to populate a large 2*D* matrix of shape (3, $N_k$)**

**Store the Cartesian components ($k_x$, $k_y$, $k_z$) for fast, vector-optimized dot products**

**Memory Footprint: Generally small for *k* points, but crucial as the foundation for the massive loop that follows**

**Step 1 is straightforward but sets up our data structures**

**Construct a loop over our mesh indices to generate the Cartesian coordinates for every single *k* point**

**Store these inside an allocated array**

**This array serves as our coordinate map, which will be read from in the next step when evaluating the phase factors of the hopping vectors**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Step 2: Evaluating and Solving $H(k)$**

**Inside a large loop running from 1 to $N_k$:**

**Construct the complex Hermitian matrix $H(k)$ using the real-space structural parameters**

**Invoke a highly optimized linear algebra routine to find the eigenvalues**

**The Workhorse Routine: LAPACK's ZHEEV (or ZHEEVD for divide-and-conquer acceleration)**

**Output: A massive array energystate($N_{band}$, $N_k$) containing all valid energy eigenvalues**

**Step 2 is where the computer spends most of its CPU cycles.**

**Run a large loop over the entire collection of *k* points. At each point, construct the tight-binding matrix. For a system like graphene with 2 atoms per unit cell and 1 orbital each, this is a simple 2×2 matrix. For more complex structures, it can grow significantly larger. Then, pass this matrix to LAPACK's ZHEEV solver, which diagonalizes it and returns a sorted list of real eigenvalues. These are stored directly into the global energy state array**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**The Physics of Hermitian Matrices in Tight Binding**

**Maintaining Numerical Integrity: The Hermitian Requirement**

**In tight binding theory, the Hamiltonian matrix must be strictly Hermitian: $H(k) = H^{\dagger}(k)$**

**Numerical Instability Alert: Small floating-point rounding errors during phase accumulation can introduce tiny anti-Hermitian components**

**The Fix: Explicitly symmetrize the matrix before passing it to the solver: $H = {}^{(H+H^{\dagger})}/_{2}$**

**Here is an important tip for implementing the code. Because quantum mechanics dictates that probabilities must be real, our Hamiltonian must be perfectly Hermitian. However, suming up millions of complex numbers ($e^{ik\cdot R}$), floating-point round-off errors can accumulate. This can leave tiny asymmetric values in the matrix, which might cause sensitive solvers to crash or generate invalid complex results. Symmetrizing the matrix explicitly using a quick half-sum line is an excellent way to maintain numerical stability**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Step 3: Statistical Binning and Sorting Mechanics**

**Define a discrete grid along the energy axis from $E_{min}$ to $E_{max}$ with a step size $\Delta E$**

**Total energy channels:** $N_E = \frac{E_{min} - E_{max}}{\Delta E}$

**For each eigenvalue $E_n(k)$, determine its matching bin index:** $Index = INT\left(\frac{E_n(k) - E_{min}}{\Delta E}\right) + 1$

**Increment the counter for that bin:** $dos_{counter}(Index) = dos_{counter}(Index) + 1$

**Once all eigenvalues are collected, use the Bin-Counting or Histogram method to counter them.**

**Divide the energy axis into uniform intervals, or bins, of width $\Delta E$. Then, loop through all of the calculated eigenvalues, use a quick truncation formula to figure out which bin each energy belongs to, and increment that bin's counter**

**This builds a raw statistical histogram of the electronic states**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Pitfalls of Raw Binning — Noise and the Need for Smoothing**

**The Problem: Raw histograms are highly sensitive to the choices made for $\Delta E$**

**If $\Delta E$ is too small: The plot becomes jagged with statistical noise**

**If $\Delta E$ is too large: Important physical details and sharp features are completely washed out**

**The Solution: Broaden the data by replacing each discrete count with a smooth Gaussian or Lorentzian curve, or implement advanced techniques like the Tetrahedron method**

**While the histogram method is easy to write, it has a major drawback: it is inherently jagged.**

**If the bin is too narrow, DOS gets wild, noisy spikes because some bins will randomly catch a few more *k* points than their neighbors. If the bin is too wide, graph looks blocky and DOS loses the Van Hove singularities.**

**To get a smooth, professional curve, broaden each data point using analytical functions like Gaussians, or use a more advanced method like the Tetrahedron integration technique**

# II. The 4 Foundational Mathematical Steps of TB-DOS calculation

**Step 4: Normalizing to Real Physical Units**

**Raw counts depend heavily on the size of your numerical grid ($N_k$). Remove this artificial dependence**

**The 3D Normalization Factor:** $$DOS(E) = \frac{Raw\ Counts(E)}{N_k \cdot \Delta E}$$

**Integrating across the entire energy range must equal the total number of orbitals in the unit cell:**

$$\int_{-\infty}^{\infty} DOS(E)dE = N_{atom} \times N_{orbitals}$$

**Step 4 ensures the plots match real physical units rather than arbitrary code parameters.**

**Raw counts scale directly with how many *k* points are chosen to sample. To make the results physically meaningful, divide by the total number of *k* points ($N_k$) and the bin width ($\Delta E$). A great way to double check the code is to integrate the final DOS curve across all energies. The total area must exactly equal the total number of active orbitals in the unit cell. If the code simulates a 2-atom graphene cell with 1 active orbital each, that integral must equal 2.0. If it does not, there is a normalization bug**

# III. Statistical Bin-Counting, Normalization and Grid Convergence Testing

**Interplay Between Bin Width and *k* mesh Density**

**Interdependence: The bin width $\Delta E$ (or smoothing width $\sigma$) is deeply tied to the *k* mesh density ($N_k$)**

**The Scaling Rule: As decreasing the bin width $\Delta E$ to capture finer physical details, one must increase the *k* point density $N_k$ to keep the bins filled consistently**

**Imbalance Consequences: A fine energy grid combined with a coarse *k* mesh creates artificial unphysical oscillations**

**When tuning the calculation, remember that the energy resolution and the momentum grid are closely linked**

**If one decides to make the energy bins narrower to see finer details, one must sample more *k* points to ensure each bin still receives enough data**

**If one uses a very fine energy grid with a sparse *k* mesh, the bins will be mostly empty, and the DOS will break apart into a series of jagged, artificial peaks that have no basis in real physics**

# IV. Projected DOS (PDOS) and Wavefunction Decomposition

**Total DOS (TDOS) tells us when states are available, but hides who is contributing**

**Real materials are built from complex orbital mixtures (*s*, *p*, *d*, *f*)**

**Use the components of the eigenvector obtained during diagonalization to break the total DOS down into specific atomic and orbital contributions**

**Now let us move into the final topic: Partial or Projected DOS, often abbreviated as PDOS. Total DOS is a fantastic global tool, but it blends all atomic contributions together into a single curve**

**If one is designing a new material, one needs to know which specific atoms and orbitals are driving the properties. By looking at the eigenvectors from the diagonalization step, one can isolate those individual contributions and see exactly how the chemistry shapes the electronic structure**

# IV. Projected DOS (PDOS) and Wavefunction Decomposition

**Mathematical Foundations of Orbital Projection**

**The full tight-binding wavefunction is a linear combination of atomic orbitals:**

$$|\Psi_n(\boldsymbol{k})\rangle = \sum_{i,\alpha} \boldsymbol{c}_{i\alpha,n}(\boldsymbol{k})|\Phi_{i\alpha}\rangle$$

**where, $i$ is the atom index, and $\alpha$ is the orbital index**

**The Probability Conservation Rule:**

$$\sum_{i,\alpha} \left|\boldsymbol{c}_{i\alpha,n}(\boldsymbol{k})\right|^2 = 1$$

**PDOS Formulation:**

$$\boldsymbol{PDOS}_{i\alpha}(\boldsymbol{E}) = \frac{1}{\boldsymbol{N}_k}\sum_{n,k} \left|\boldsymbol{c}_{i\alpha,n}(\boldsymbol{k})\right|^2 \boldsymbol{\delta}[\boldsymbol{E} - \boldsymbol{E}_n(\boldsymbol{k})]$$

**Let us look at the underlying math. When LAPACK solves the eigenvalue problem, it gives both the energy $E_n(k)$ and the eigenvector coefficients $c_{i\alpha,n}(k)$. These coefficients represent the probability amplitudes for finding an electron in orbital $\alpha$ on atom $i$. Because the total wavefunction is normalized, the sum of their squares equals 1. To calculate the PDOS for a specific orbital, simply weight the statistical counting by the probability $\left|c_{i\alpha,n}(k)\right|^2$ instead of adding a flat 1 to our bin. This scales the contribution based on that orbital's actual presence in the state**

# IV. Projected DOS (PDOS) and Wavefunction Decomposition

**PDOS Case Study: $\pi$ and $\pi^*$ Bands in Graphene**

**Graphene's planar structure is stabilized by $sp^2$ hybridized orbitals ($s$, $p_x$, $p_y$)**

**The out-of-plane $p_z$ orbitals remain unhybridized, forming weak $\pi$ bonds**

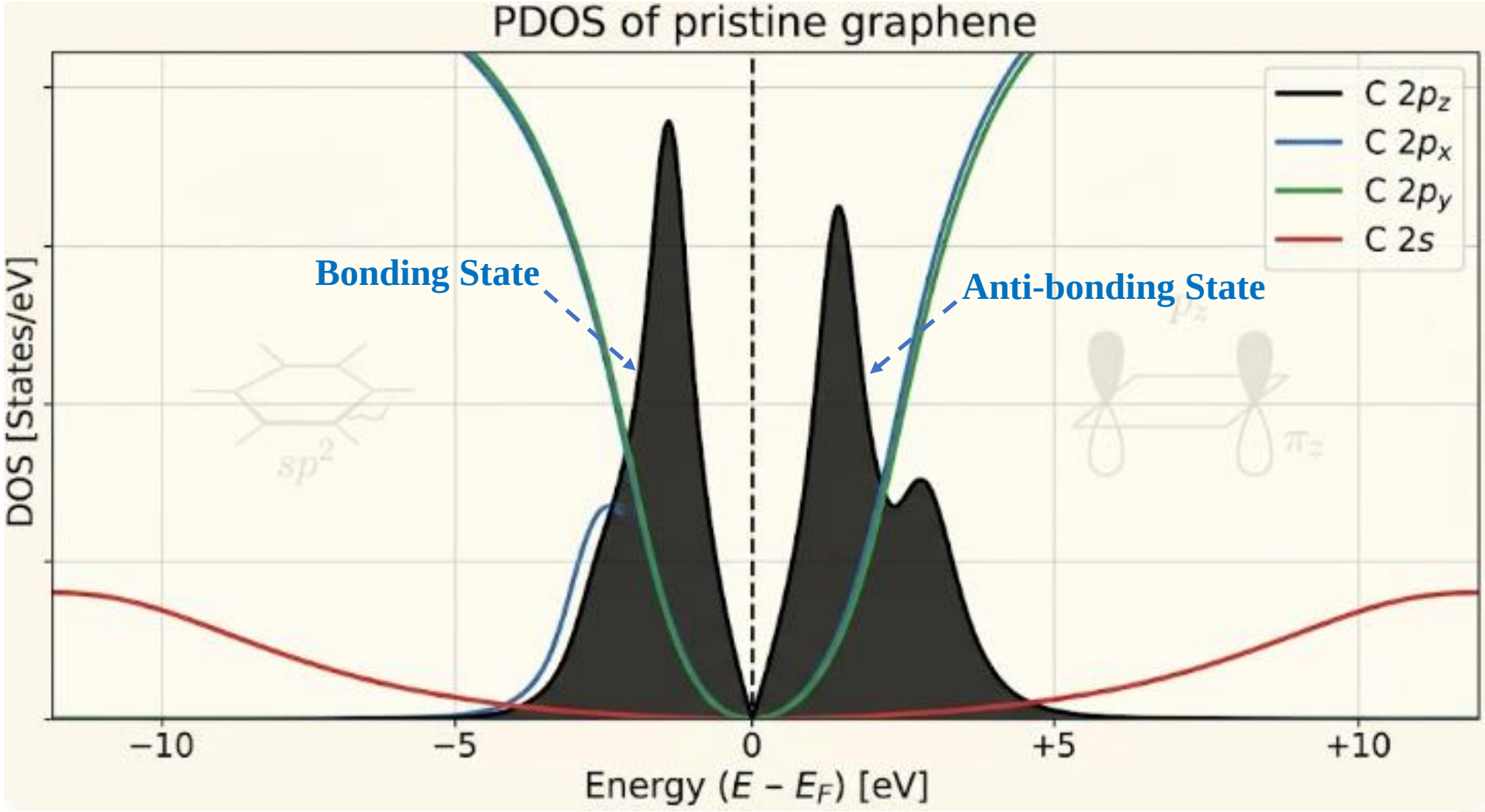
**PDOS Analysis: Projecting the DOS specifically onto the Carbon $\pi$ state**

**Result: The signature *V*-shaped DOS profile near the Fermi level is driven entirely by the $p_z$ orbitals, while the $sp^2$ states are buried much deeper in energy**

**Let us apply this to a classic system: Graphene**

**Graphene's carbon atoms use $s$, $p_x$ and $p_y$ orbitals to form strong covalent bonds within the sheet. The remaining $p_z$ orbitals point straight out of the plane. Running a tight-binding calculation and map out the PDOS for the $p_z$ orbitals, one will find that they map perfectly to the *V*-shaped curve around the Fermi level. This proves visually that graphene's low-energy conductivity is entirely governed by these out-of-plane $\pi$ bonds, while the structural $sp^2$ bonds are nestled safely deep within the valence bands**

# IV. Projected DOS (PDOS) and Wavefunction Decomposition



# IV. Projected DOS (PDOS) and Wavefunction Decomposition

**Interpreting Bonding Profiles through PDOS Overlap:**

**Bonding State: PDOS peaks for two different atoms overlap perfectly within the same energy window below the Fermi level**

**Anti-bonding State: PDOS peaks overlap at higher energies above the Fermi level**

**Non-bonding State: An isolated peak appears on one atom's PDOS with no corresponding partner peak on adjacent atoms**

**PDOS is incredibly valuable because it shows read chemical bonding directly from an energy plot. Peaks from two different atoms aligning perfectly at the same energy below the Fermi level, indicates a stable bonding state where their wavefunctions hybridize. If they align above the Fermi level, an anti-bonding state shows up. If a peak shows up on one atom but is completely flat on the others, it represents an isolated, non-bonding lone pair.**

**This provides a direct way to translate raw quantum mechanics into classic chemical concepts**

# IV. Projected DOS (PDOS) and Wavefunction Decomposition

**Algorithmic Considerations for Scale and Efficiency**

**The Bottleneck: Diagonalizing matrices scales poorly as $O(M^3)$, where $M$ is the matrix dimension**

**Exploiting Sparsity: Tight-binding matrices are naturally sparse since atoms only interact with nearby neighbors**

**Advanced Solvers: For large multi-atom cells, shift from full diagonalization (ZHEEV) to iterative Krylov-subspace methods (*e.g.*, Lanczos or Kernel Polynomial Methods) to find the DOS directly without computing explicit eigenvectors**

**As preparing to implement codes, one needs to think about efficiency. While full diagonalization works beautifully for small systems like graphene, it struggles with larger structures because the computational cost scales cubically $O(M^3)$ with matrix size. Scaling up to a large supercell or a complex moiré lattice, full diagonalization will quickly overwhelm the hardware. To handle larger models, one can take advantage of the fact that tight binding matrices are highly sparse, allowing iterative techniques like the Lanczos or Kernel Polynomial methods to compute the DOS directly while bypassing the costly diagonalization loop entirely**



**That's all for this session**

**Thank you for your attention and great participation**